

1. INTRODUCCIÓN

El presente informe describe el desarrollo e implementación de un Sistema de Biblioteca, desarrollado utilizando el lenguaje de programación Go (Golang) y conectado al sistema gestor de bases de datos Microsoft SQL Server.

El sistema fue desarrollado como una aplicación de consola y permite realizar diferentes operaciones relacionadas con la administración de una biblioteca, entre ellas el registro de estudiantes, registro de libros, préstamos, devoluciones y consulta de información.

Durante el desarrollo se configuró el entorno de programación, se creó el módulo de Go, se instaló el controlador `go-mssqldb` y se estableció una conexión con SQL Server mediante el puerto **1433**.

Además, se realizaron pruebas de funcionamiento para comprobar la correcta ejecución de las principales funciones del sistema.

2. OBJETIVOS

2.1 Objetivo general

Desarrollar un sistema de gestión de biblioteca utilizando Go y SQL Server, que permita administrar estudiantes, libros y préstamos mediante una aplicación de consola.

2.2 Objetivos específicos

- Desarrollar una aplicación utilizando el lenguaje Go.
- Crear estructuras para representar estudiantes, libros y préstamos.
- Configurar un proyecto de Go mediante módulos.
- Instalar y utilizar el controlador de Microsoft SQL Server para Go.
- Establecer una conexión entre Go y SQL Server.
- Registrar estudiantes y comprobar su almacenamiento en la base de datos.
- Implementar el registro y consulta de libros.
- Implementar préstamos y devoluciones de libros.
- Incorporar validaciones para controlar los datos ingresados.
- Realizar pruebas de funcionamiento del sistema.

3. HERRAMIENTAS UTILIZADAS

Para el desarrollo del proyecto se utilizaron las siguientes herramientas:

Herramienta	Función
Go (Golang)	Lenguaje utilizado para desarrollar el sistema
Visual Studio Code	Edición del código fuente
Microsoft SQL Server	Gestión de la base de datos
SQL Server Management Studio	Administración y consulta de SQL Server
PowerShell	Ejecución del programa y comandos de Go
go-mssqldb	Controlador para conectar Go con SQL Server

4. CONFIGURACIÓN DEL PROYECTO

El proyecto se creó dentro de la carpeta:

`C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca`

El archivo principal de la aplicación es:

`Biblioteca.go`

Para inicializar el proyecto se ejecutó:

`go mod init biblioteca`

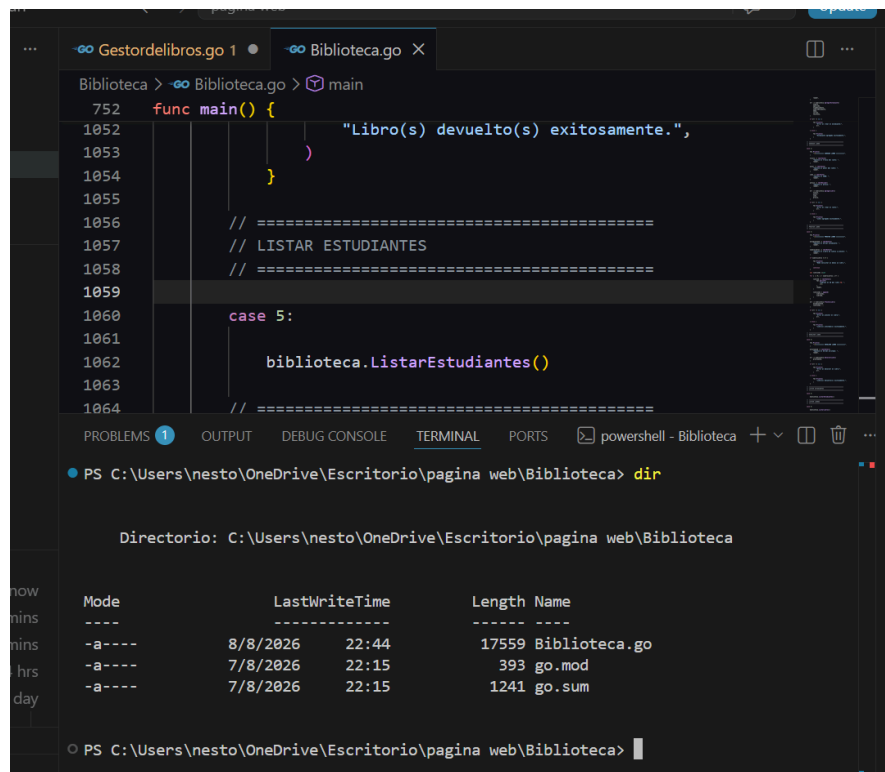
Posteriormente se instaló el controlador de SQL Server mediante:

`go get github.com/microsoft/go-mssqldb`

Finalmente, el código fue formateado utilizando:

```
gofmt -w Biblioteca.go
```

Figura 1. Configuración del proyecto Go



```
Biblioteca > go Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054
1055 // =====
1056 // LISTAR ESTUDIANTES
1057 // =====
1058
1059 case 5:
1060     biblioteca.ListarEstudiantes()
1061
1062 // =====
1063
1064
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - Biblioteca + - ...

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> dir

Directorio: C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca

Mode	LastWriteTime	Length	Name
-a----	8/8/2026 22:44	17559	Biblioteca.go
-a----	7/8/2026 22:15	393	go.mod
-a----	7/8/2026 22:15	1241	go.sum

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca>

5. CONFIGURACIÓN DE SQL SERVER

Para el proyecto se utilizó una base de datos denominada:

Biblioteca

La aplicación se configuró para conectarse al servidor local mediante el puerto:

1433

La cadena de conexión utilizada en el programa es:

cadena :=

```
"server=localhost;port=1433;database=Biblioteca;trusted_connection=yes;TrustServerCertificate=true"
```

La conexión se realiza mediante el paquete:

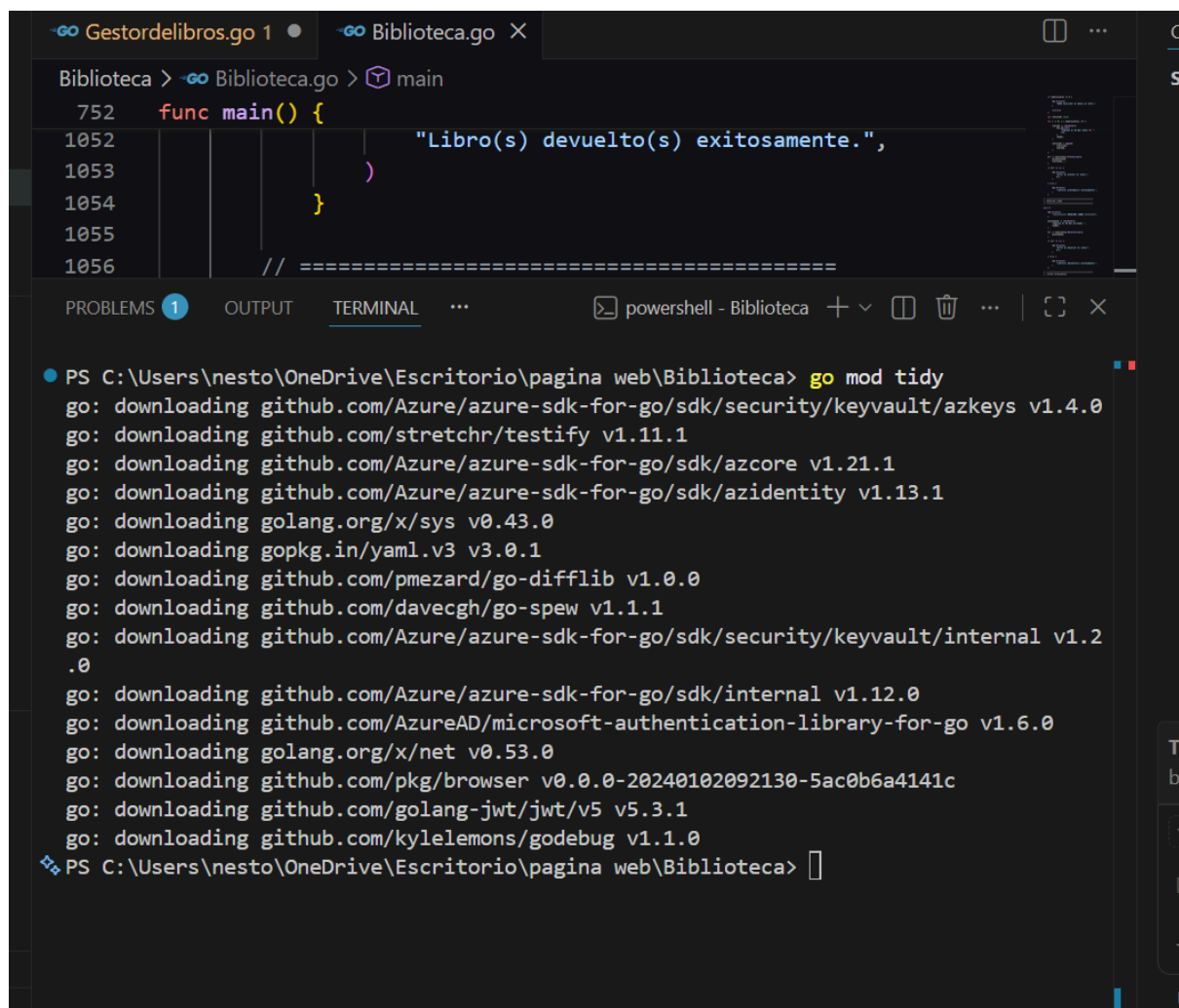
`database/sql`

y el controlador:

`github.com/microsoft/go-mssqldb`

La función `db.Ping()` permite comprobar que la conexión con SQL Server se encuentre disponible.

Figura 2. Configuración del puerto de SQL Server



The screenshot shows a Go IDE with two tabs: 'Gestordelibros.go' and 'Biblioteca.go'. The 'Biblioteca.go' tab is active, showing a Go function `func main()` with line numbers 752 to 1056. The function body includes a string `"Libro(s) devuelto(s) exitosamente."` and a comment `// =====`. Below the code editor is a terminal window titled 'powershell - Biblioteca'. The terminal shows the command `go mod tidy` being executed, which results in a list of dependencies being downloaded, including `github.com/Azure/azure-sdk-for-go/sdk/security/keyvault/azkeys v1.4.0`, `github.com/stretchr/testify v1.11.1`, `github.com/Azure/azure-sdk-for-go/sdk/azcore v1.21.1`, `github.com/Azure/azure-sdk-for-go/sdk/azidentity v1.13.1`, `golang.org/x/sys v0.43.0`, `gopkg.in/yaml.v3 v3.0.1`, `github.com/pmezard/go-difflib v1.0.0`, `github.com/davecgh/go-spew v1.1.1`, `github.com/Azure/azure-sdk-for-go/sdk/security/keyvault/internal v1.2.0`, `github.com/Azure/azure-sdk-for-go/sdk/internal v1.12.0`, `github.com/AzureAD/microsoft-authentication-library-for-go v1.6.0`, `golang.org/x/net v0.53.0`, `github.com/pkg/browser v0.0.0-20240102092130-5ac0b6a4141c`, `github.com/golang-jwt/jwt/v5 v5.3.1`, and `github.com/kylelemons/godebug v1.1.0`.

LocalAddress LocalPort

:: 1433

0.0.0.0 1433

Esto demuestra que SQL Server está escuchando en el puerto 1433.

6. CONEXIÓN ENTRE GO Y SQL SERVER

La aplicación realiza la conexión a SQL Server mediante la función `conectarBD()`.

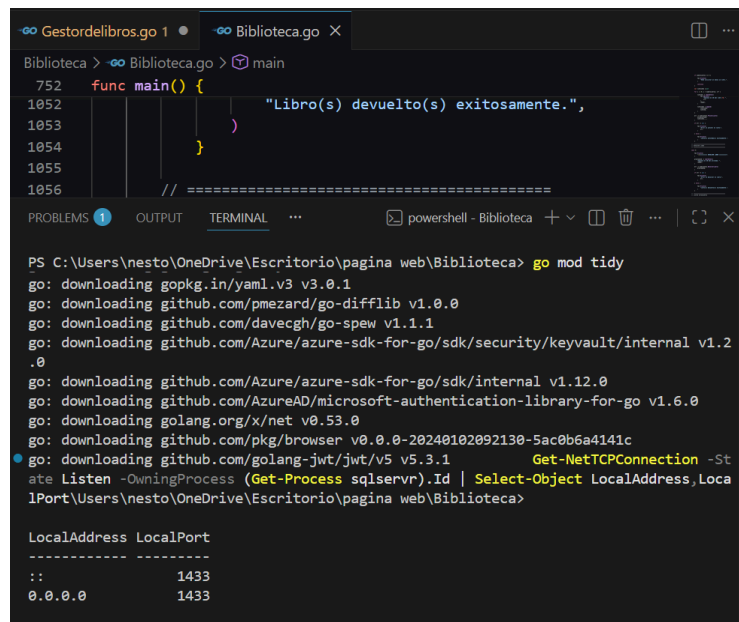
El procedimiento consiste en:

1. Crear la cadena de conexión.
2. Abrir la conexión mediante `sql.Open()`.
3. Comprobar la conexión mediante `db.Ping()`.
4. Mostrar un mensaje indicando que la conexión fue exitosa.

Cuando la conexión es correcta, el programa muestra:

Conexión con SQL Server exitosa.

Figura 3. Conexión exitosa con SQL Server



```

Biblioteca > Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ... powershell - Biblioteca + - - - - -

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go mod tidy
go: downloading gopkg.in/yaml.v3 v3.0.1
go: downloading github.com/pmezard/go-difflib v1.0.0
go: downloading github.com/davecgh/go-spew v1.1.1
go: downloading github.com/Azure/azure-sdk-for-go/sdk/security/keyvault/internal v1.2.0
go: downloading github.com/Azure/azure-sdk-for-go/sdk/internal v1.12.0
go: downloading github.com/AzureAD/microsoft-authentication-library-for-go v1.6.0
go: downloading golang.org/x/net v0.53.0
go: downloading github.com/pkg/browser v0.0.0-20240102092130-5ac0b6a4141c
go: downloading github.com/golang-jwt/jwt/v5 v5.3.1
Get-NetTCPConnection -State Listen -OwningProcess (Get-Process sqlservr).Id | Select-Object LocalAddress,LocalPort\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca>

LocalAddress LocalPort
-----
:: 1433
0.0.0.0 1433
```

Conexión con SQL Server exitosa.

7. ESTRUCTURA DEL SISTEMA

El programa utiliza tres estructuras principales:

7.1 Estructura Estudiante

La estructura **Estudiante** contiene los siguientes atributos:

- ID.
- Nombre.
- Apellido.
- Tipo de documento.
- Número de documento.
- Edad.
- Correo.
- Teléfono.

Para cada atributo se implementaron métodos **Get** y **Set**, permitiendo consultar y modificar la información.

7.2 Estructura Libro

La estructura **Libro** contiene:

- ID.
- Título.
- Autor.
- ISBN.
- Precio.
- Estado del préstamo.

El atributo **prestado** permite determinar si el libro está disponible o actualmente prestado.

7.3 Estructura Prestamo

La estructura **Prestamo** contiene:

- ID del préstamo.
- Estudiante asociado.
- Libros prestados.
- Fecha de préstamo.

- Fecha de devolución.

Esto permite relacionar un estudiante con uno o varios libros.

8. MENÚ PRINCIPAL

Al ejecutar el programa se presenta el siguiente menú:

```
=====

SISTEMA DE BIBLIOTECA

=====

1. Agregar estudiante

2. Agregar libro

3. Prestar libro

4. Devolver libro

5. Listar estudiantes

6. Listar libros

7. Listar prestamos

8. Salir

=====
```

Seleccione una opción:

El menú permite al usuario seleccionar la operación que desea realizar.

Figura 4. Menú principal del sistema

The screenshot shows a VS Code editor with two tabs: 'Gestordelibros.go 1' and 'Biblioteca.go'. The 'Biblioteca.go' tab is active, showing a Go program with a `main` function. The code includes a `func main()` block with a `println` statement: `println("Libro(s) devuelto(s) exitosamente.",`. Below the code, the 'TERMINAL' panel shows the execution of the program. The terminal output includes the command `Get-NetTCPConnection -State Listen -OwningProcess (Get-Process sqlservr).Id | Select-Object LocalAddress, LocalPort`, which returns two lines of output: `:: 1433` and `0.0.0.0 1433`. Below this, the command `go run Biblioteca.go` is executed, resulting in the output: `Conexión con SQL Server exitosa.` followed by a menu titled `SISTEMA DE BIBLIOTECA` with eight options: `1. Agregar estudiante`, `2. Agregar libro`, `3. Prestar libro`, `4. Devolver libro`, `5. Listar estudiantes`, `6. Listar libros`, `7. Listar prestamos`, and `8. Salir`. The prompt `Seleccione una opción:` is shown at the bottom.

9. AGREGAR ESTUDIANTE

La opción **1. Agregar estudiante** permite registrar los datos de un estudiante.

El sistema solicita:

- Nombre.
- Apellido.
- Tipo de documento.
- Número de documento.
- Edad.
- Correo.
- Teléfono.

El programa también valida que los campos obligatorios no estén vacíos y que la edad sea mayor que cero.

Durante las pruebas se registró un estudiante con los datos:

Nombre: Paula

Apellido: Arcos

Tipo de documento: cedula

Número de documento: 1725029779

Edad: 22

Correo: paula.daniela@hotmail.com

Teléfono: 0984494009

El sistema mostró:

Estudiante agregado exitosamente.

Figura 5. Registro de estudiante

```
7. Listar prestatarios
8. Salir
=====
Seleccione una opción: 1

===== AGREGAR ESTUDIANTE =====
Ingrese el nombre: Paula
Ingrese el apellido: Arcos
Ingrese el tipo de documento: cedula
Ingrese el número de documento: 17125029779
Ingrese la edad: 22
Ingrese el correo: paula.daniela@hotmail.com
Ingrese el teléfono: 0984494009
Estudiante agregado exitosamente.
```

10. VERIFICACIÓN DEL ESTUDIANTE EN SQL SERVER

Para comprobar que el estudiante fue almacenado correctamente en la base de datos, se ejecutó la siguiente consulta:

USE Biblioteca;

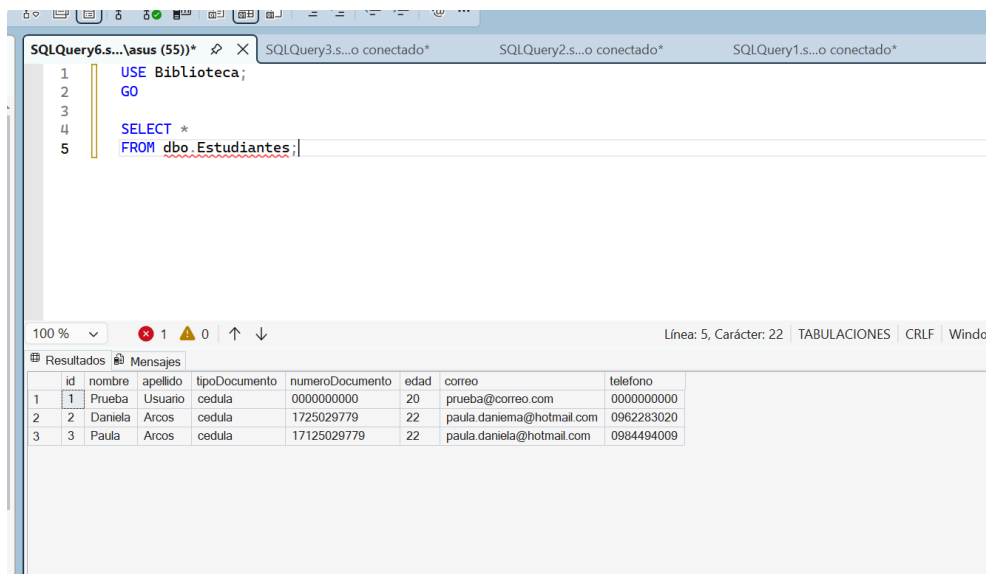
GO

```
SELECT *
```

```
FROM dbo.Estudiantes;
```

El resultado permitió comprobar que los datos registrados desde la aplicación se encontraban almacenados en SQL Server.

Figura 6. Estudiante almacenado en SQL Server



Esta prueba permite demostrar la comunicación entre la aplicación desarrollada en Go y la base de datos.

11. AGREGAR LIBRO

La opción **2. Agregar libro** permite registrar un nuevo libro.

El programa solicita:

- Título.
- Autor.
- ISBN.
- Precio.

Como prueba se puede registrar:

Título: El Principito

ID

Título

Autor

ISBN

Precio

Estado

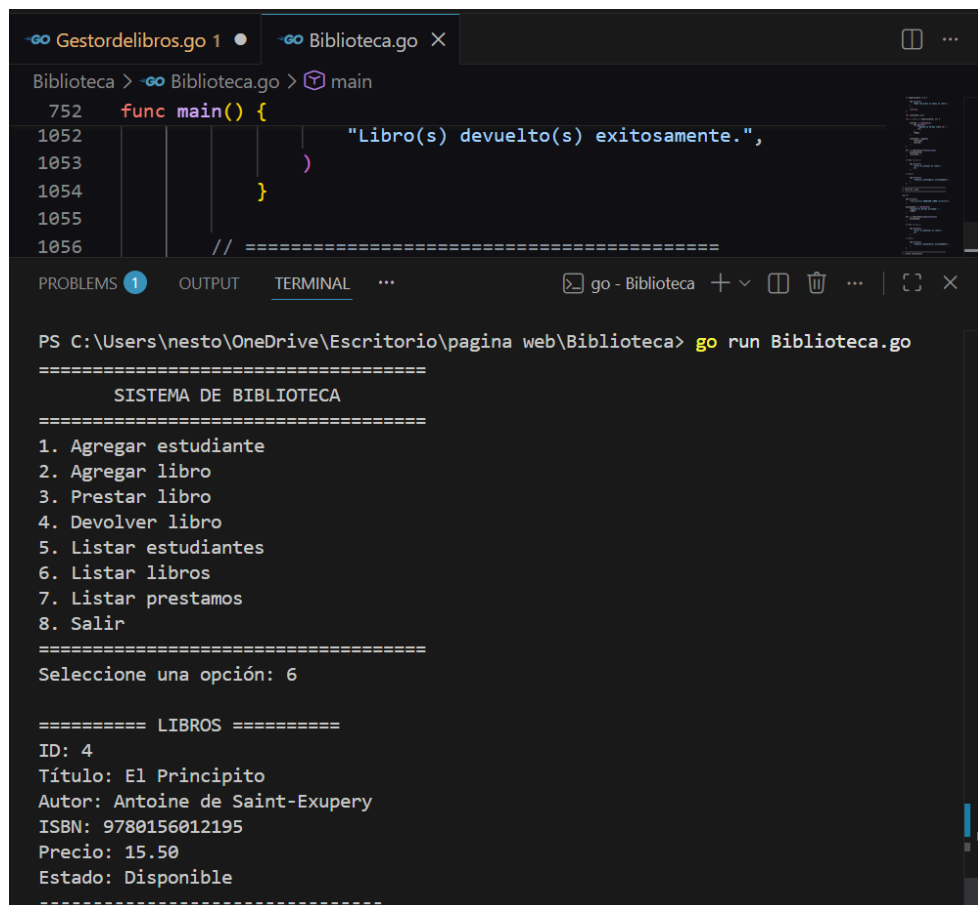
El estado puede ser:

Disponible

o:

Prestado

Figura 8. Listado de libros



```

Biblioteca > Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ...
PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go
=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 6

===== LIBROS =====
ID: 4
Título: El Principito
Autor: Antoine de Saint-Exupery
ISBN: 9780156012195
Precio: 15.50
Estado: Disponible
=====
```

13. PRESTAR LIBRO

La opción **3. Prestar libro** permite asignar uno o varios libros a un estudiante.

El sistema solicita:

1. ID del estudiante.
2. Número de libros.
3. ID de cada libro.

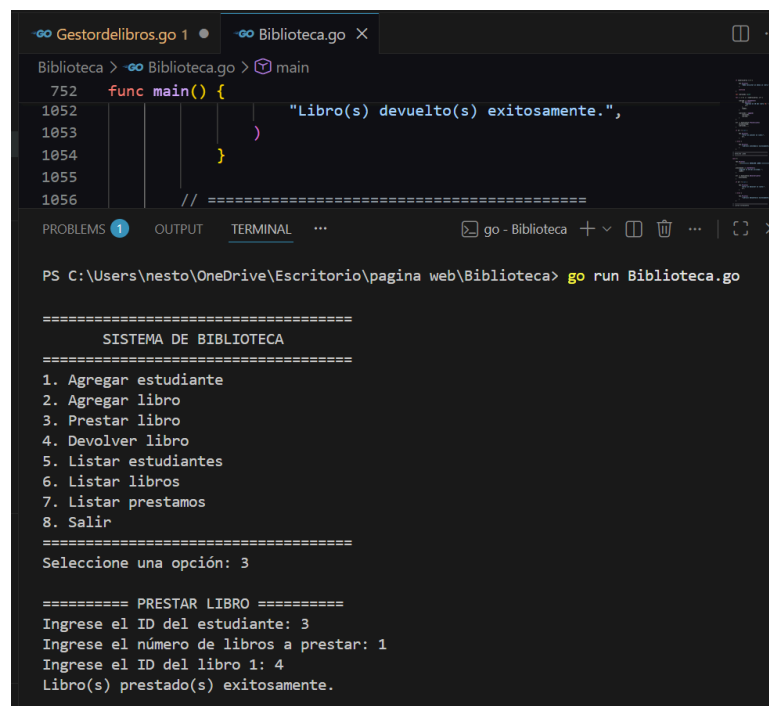
Antes de realizar el préstamo, se comprueba que:

- El estudiante exista.
- El libro exista.
- El libro no esté actualmente prestado.
- Se haya seleccionado al menos un libro.

Cuando la operación es correcta se muestra:

Libro(s) prestado(s) exitosamente.

Figura 9. Préstamo de libro



```
gestordelibros.go 1 Biblioteca.go X
Biblioteca > - Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ... go - Biblioteca + - - - - -
PS C:\Users\neato\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go

=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 3

===== PRESTAR LIBRO =====
Ingrese el ID del estudiante: 3
Ingrese el número de libros a prestar: 1
Ingrese el ID del libro 1: 4
Libro(s) prestado(s) exitosamente.
```

14. COMPROBACIÓN DEL ESTADO DEL LIBRO

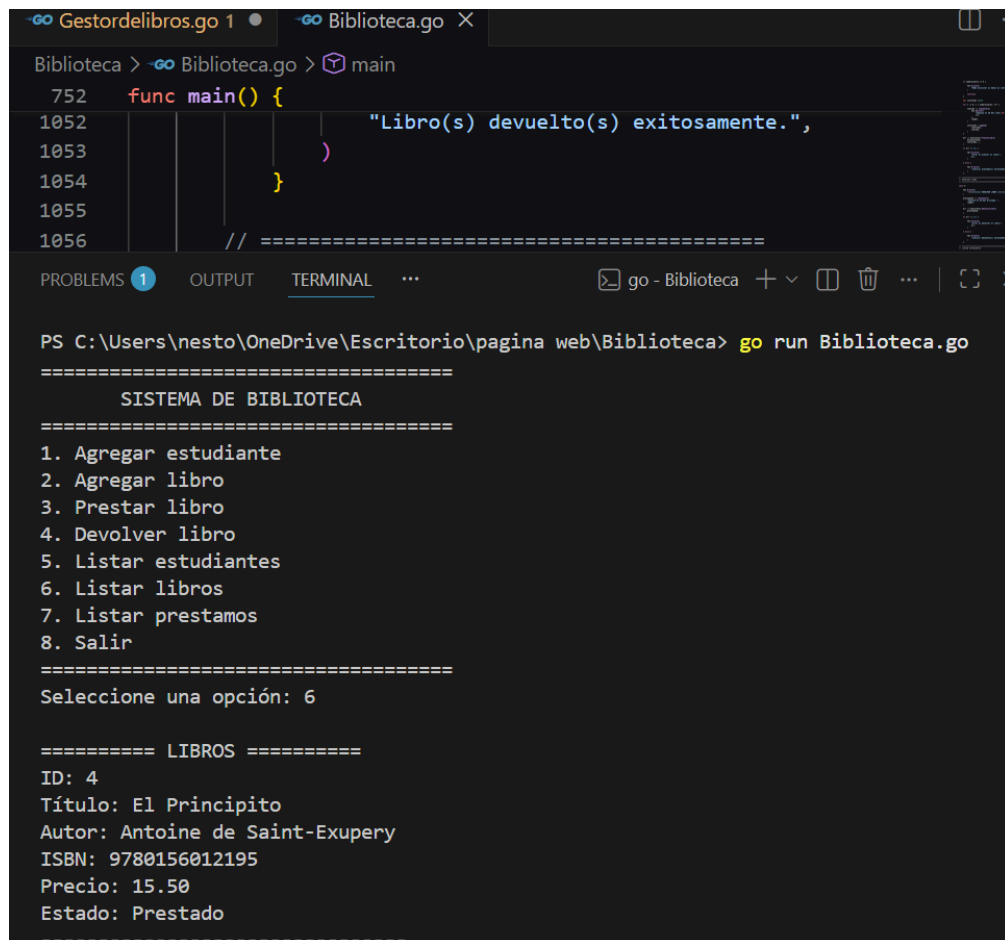
Después de realizar el préstamo, se seleccionó la opción **6. Listar libros**.

El libro utilizado en la prueba cambió su estado de:

Disponible a: Prestado

Esto permite comprobar que el sistema controla correctamente el estado de disponibilidad del libro.

Figura 10. Libro en estado prestado



```

Biblioteca > go run Biblioteca.go
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 }
1056 // =====

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go
=====
          SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 6

===== LIBROS =====
ID: 4
Título: El Principito
Autor: Antoine de Saint-Exupery
ISBN: 9780156012195
Precio: 15.50
Estado: Prestado
=====
```

15. LISTAR PRÉSTAMOS

La opción **7. Listar prestamos** permite consultar los préstamos registrados.

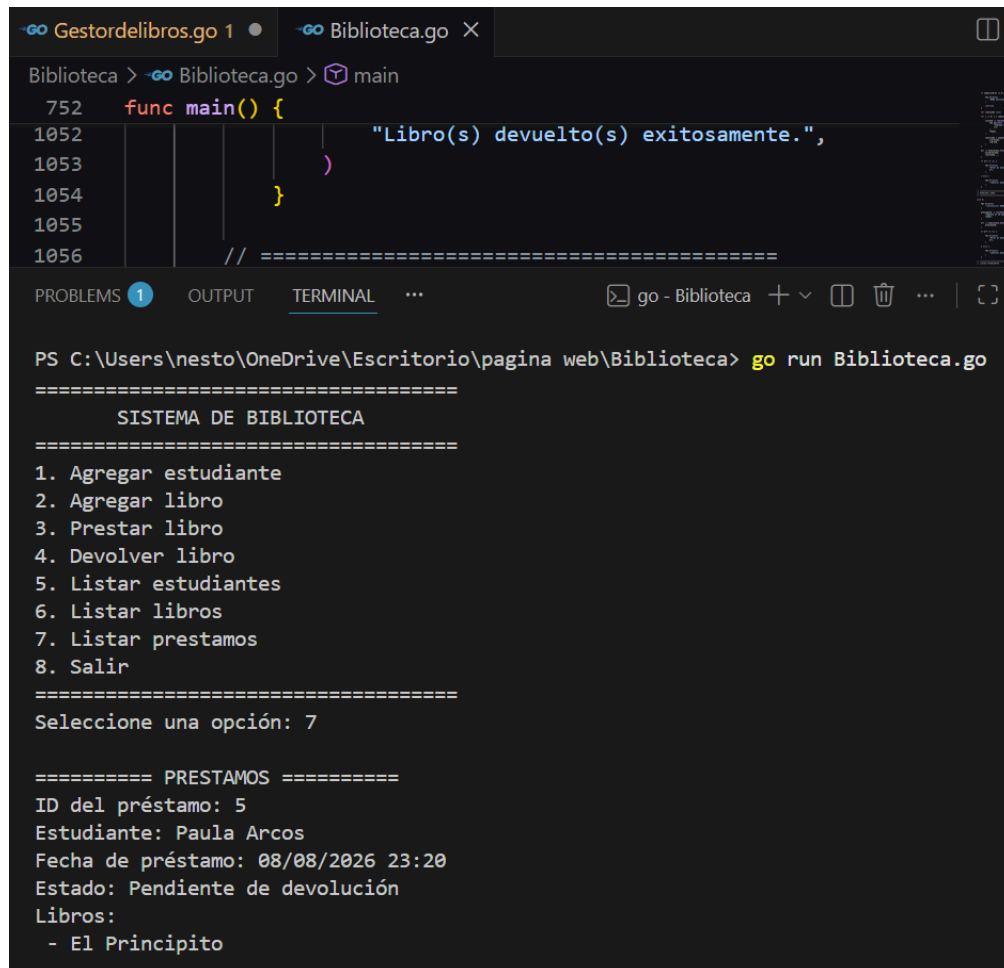
La información mostrada incluye:

- ID del préstamo.
- Estudiante.
- Fecha de préstamo.
- Estado.
- Libros asociados.

Cuando un préstamo todavía no ha sido devuelto, aparece:

Estado: Pendiente de devolución

Figura 11. Listado de préstamos



```
go Gestordelibros.go 1 • go Biblioteca.go X
Biblioteca > -go Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ... go - Biblioteca + - [ ] [ ] [ ] [ ]

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go

=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 7

===== PRESTAMOS =====
ID del préstamo: 5
Estudiante: Paula Arcos
Fecha de préstamo: 08/08/2026 23:20
Estado: Pendiente de devolución
Libros:
- El Principito
=====
```

16. DEVOLVER LIBRO

La opción **4. Devolver libro** permite registrar la devolución de los libros asociados a un préstamo.

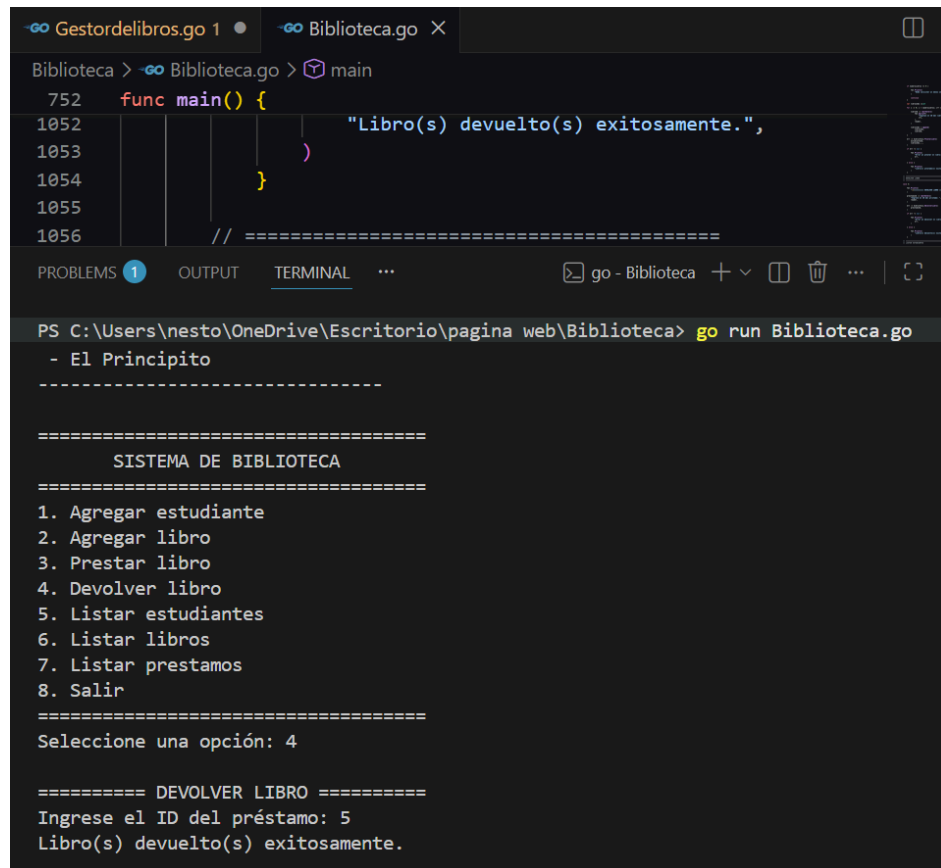
El usuario debe introducir el ID del préstamo.

Cuando la devolución es correcta se muestra:

Libro(s) devuelto(s) exitosamente.

El programa registra la fecha de devolución y cambia nuevamente el estado de los libros a disponibles.

Figura 12. Devolución de libro



```
Biblioteca > Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056
```

```
PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go
- El Principito
-----

=====
          SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 4

===== DEVOLVER LIBRO =====
Ingrese el ID del préstamo: 5
Libro(s) devuelto(s) exitosamente.
```

17. COMPROBACIÓN DE DEVOLUCIÓN

Después de realizar la devolución, se seleccionó nuevamente:

6. Listar libros

El libro utilizado en la prueba volvió a mostrar:

Estado: Disponible

Esto permite verificar que el sistema actualiza correctamente el estado del libro después de una devolución.

Figura 13. Libro disponible después de la devolución

```

Gestordelibros.go 1  Biblioteca.go X
Biblioteca > Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ... go - Biblioteca + - - - - -

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go

=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 6

===== LIBROS =====
ID: 4
Título: El Principito
Autor: Antoine de Saint-Exupery
ISBN: 9780156012195
Precio: 15.50
Estado: Disponible
-----
```

```

Gestordelibros.go 1  Biblioteca.go X
Biblioteca > Biblioteca.go > main
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056

PROBLEMS 1 OUTPUT TERMINAL ... go - Biblioteca + - - - - -

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go

=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 7

===== PRESTAMOS =====
ID del préstamo: 5
Estudiante: Paula Arcos
Fecha de préstamo: 08/08/2026 23:20
Fecha de devolución: 08/08/2026 23:25
Libros:
- El Principito
-----
```

18. VALIDACIÓN DE ERRORES

El programa cuenta con mecanismos para controlar situaciones incorrectas.

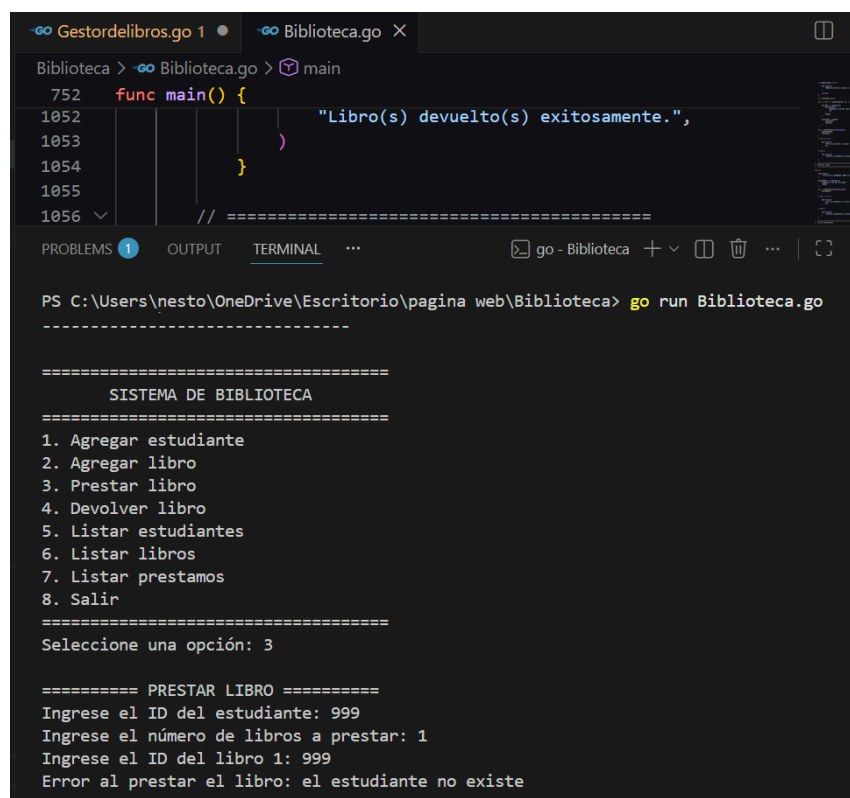
Por ejemplo, si se intenta prestar un libro utilizando un estudiante inexistente, el sistema muestra:

Error al prestar el libro: el estudiante no existe

También se controla la existencia de los libros y que un libro no pueda ser prestado nuevamente mientras ya se encuentre prestado.

Además, las funciones de lectura permiten controlar la introducción de valores numéricos.

Figura 14. Validación de errores



The screenshot shows a Go IDE with two tabs: 'Gestordelibros.go 1' and 'Biblioteca.go'. The 'Biblioteca.go' tab is active, showing a Go program with a `main` function. The program is running in a terminal window. The terminal output shows a menu of options for a library system. Option 3, 'Prestar libro', is selected. The program prompts for the student ID (999), the number of books to lend (1), and the book ID (999). It then displays the error message: 'Error al prestar el libro: el estudiante no existe'.

```
752 func main() {
1052     "Libro(s) devuelto(s) exitosamente.",
1053 }
1054 }
1055 // =====
1056 // =====

PS C:\Users\nesto\OneDrive\Escritorio\pagina web\Biblioteca> go run Biblioteca.go

=====
SISTEMA DE BIBLIOTECA
=====
1. Agregar estudiante
2. Agregar libro
3. Prestar libro
4. Devolver libro
5. Listar estudiantes
6. Listar libros
7. Listar prestamos
8. Salir
=====
Seleccione una opción: 3

===== PRESTAR LIBRO =====
Ingrese el ID del estudiante: 999
Ingrese el número de libros a prestar: 1
Ingrese el ID del libro 1: 999
Error al prestar el libro: el estudiante no existe
```

19. FUNCIONES IMPLEMENTADAS

El sistema cuenta con las siguientes funciones principales:

Función	Descripción
---------	-------------

conectarBD()	Establece la conexión con SQL Server
AgregarEstudiante()	Registra estudiantes
AgregarLibro()	Registra libros
PrestarLibro()	Registra préstamos
DevolverLibro()	Registra devoluciones
ListarEstudiantes()	Muestra estudiantes
ListarLibros()	Muestra libros
ListarPrestamos()	Muestra préstamos
leerTexto()	Lee datos de texto
leerEntero()	Lee números enteros
leerDecimal()	Lee valores decimales

20. RESULTADOS OBTENIDOS

El desarrollo permitió obtener una aplicación funcional para la administración básica de una biblioteca.

Se consiguió configurar correctamente el entorno de Go y establecer comunicación con SQL Server mediante el controlador `go-mssqldb`.

Durante las pruebas se comprobó que el sistema permite registrar estudiantes, administrar libros, realizar préstamos y devoluciones y consultar la información desde la aplicación.

Además, se comprobó directamente en SQL Server que los datos de estudiantes registrados desde la aplicación pueden almacenarse y consultarse mediante instrucciones SQL.

22. DIFICULTADES PRESENTADAS

Durante la implementación se presentó inicialmente un problema relacionado con la conexión a SQL Server.

El sistema mostraba el mensaje:

Error al conectar con SQL Server:

no instance matching 'SQLEXPRESS' returned from host 'localhost'

Para solucionar el problema se verificó el proceso `sqlservr`, el puerto utilizado por SQL Server y la configuración de los protocolos.

Finalmente se comprobó que SQL Server estaba escuchando correctamente en el puerto:

1433

Posteriormente se modificó la cadena de conexión de Go para utilizar:

`server=localhost;port=1433`

Después de realizar estos cambios se obtuvo:

Conexión con SQL Server exitosa.

Esto permitió continuar con las pruebas del sistema.

23. CONCLUSIONES

1. Se desarrolló un sistema de biblioteca utilizando el lenguaje de programación Go.

2. Se logró establecer una conexión funcional entre Go y Microsoft SQL Server mediante el controlador `go-mssqldb`.
3. Se comprobó que la aplicación puede registrar información de estudiantes y verificarla posteriormente en SQL Server.
4. Se implementaron estructuras para representar estudiantes, libros y préstamos.
5. Se desarrollaron funcionalidades para registrar libros, realizar préstamos y procesar devoluciones.
6. Se incorporaron validaciones para controlar errores y evitar operaciones incorrectas.
7. Las pruebas realizadas permitieron comprobar el funcionamiento de las principales opciones del sistema.
8. El proyecto permitió aplicar conocimientos relacionados con programación, estructuras de datos, manejo de errores, bases de datos y conexión entre una aplicación y un sistema gestor de bases de datos.

24. RECOMENDACIONES

Como futuras mejoras del sistema se recomienda:

- Implementar una interfaz gráfica o aplicación web.
- Incorporar autenticación de usuarios.
- Implementar búsqueda de estudiantes y libros.
- Permitir actualizar y eliminar registros.
- Implementar un sistema de multas por retrasos.
- Generar reportes de préstamos.
- Implementar copias de seguridad automáticas.
- Mejorar la persistencia de todas las operaciones directamente en SQL Server.
- Implementar relaciones y restricciones de integridad en la base de datos.